

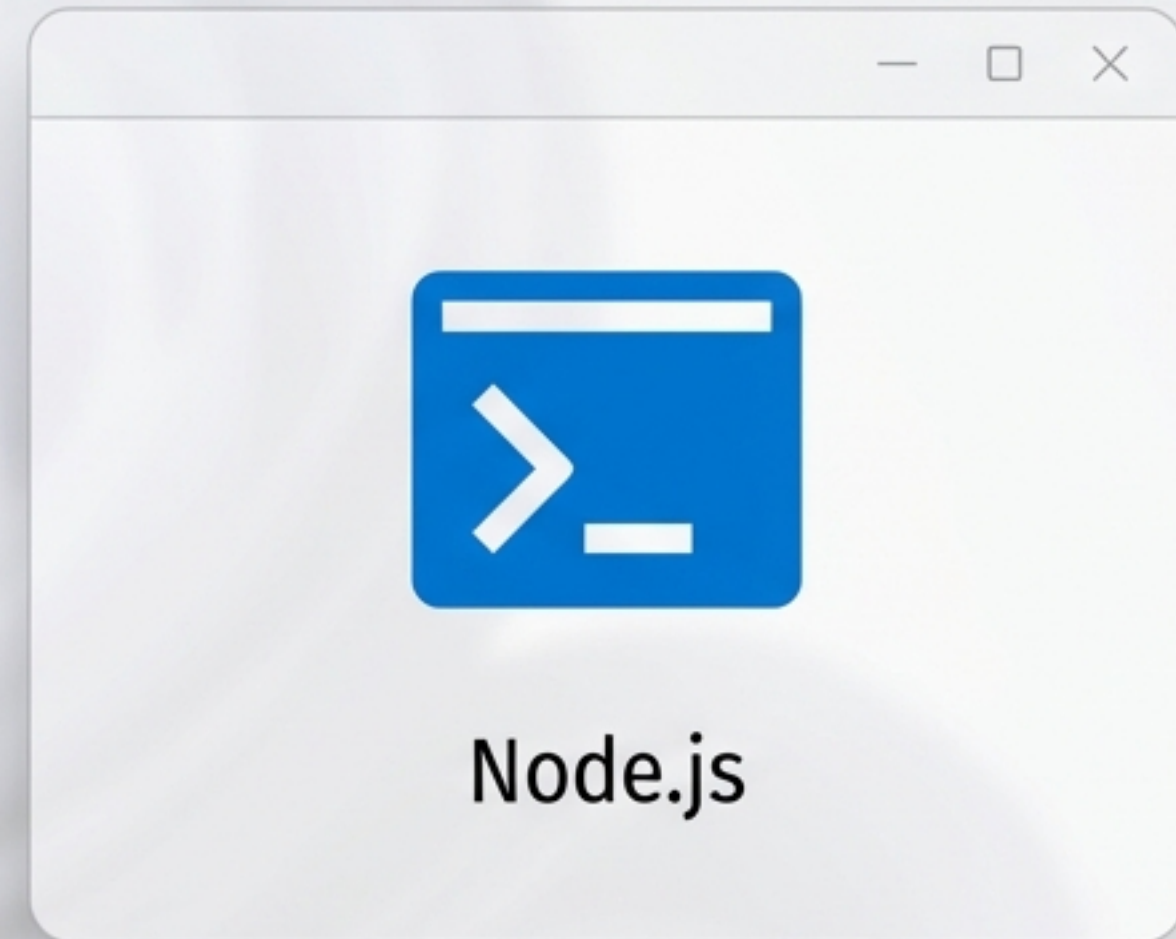
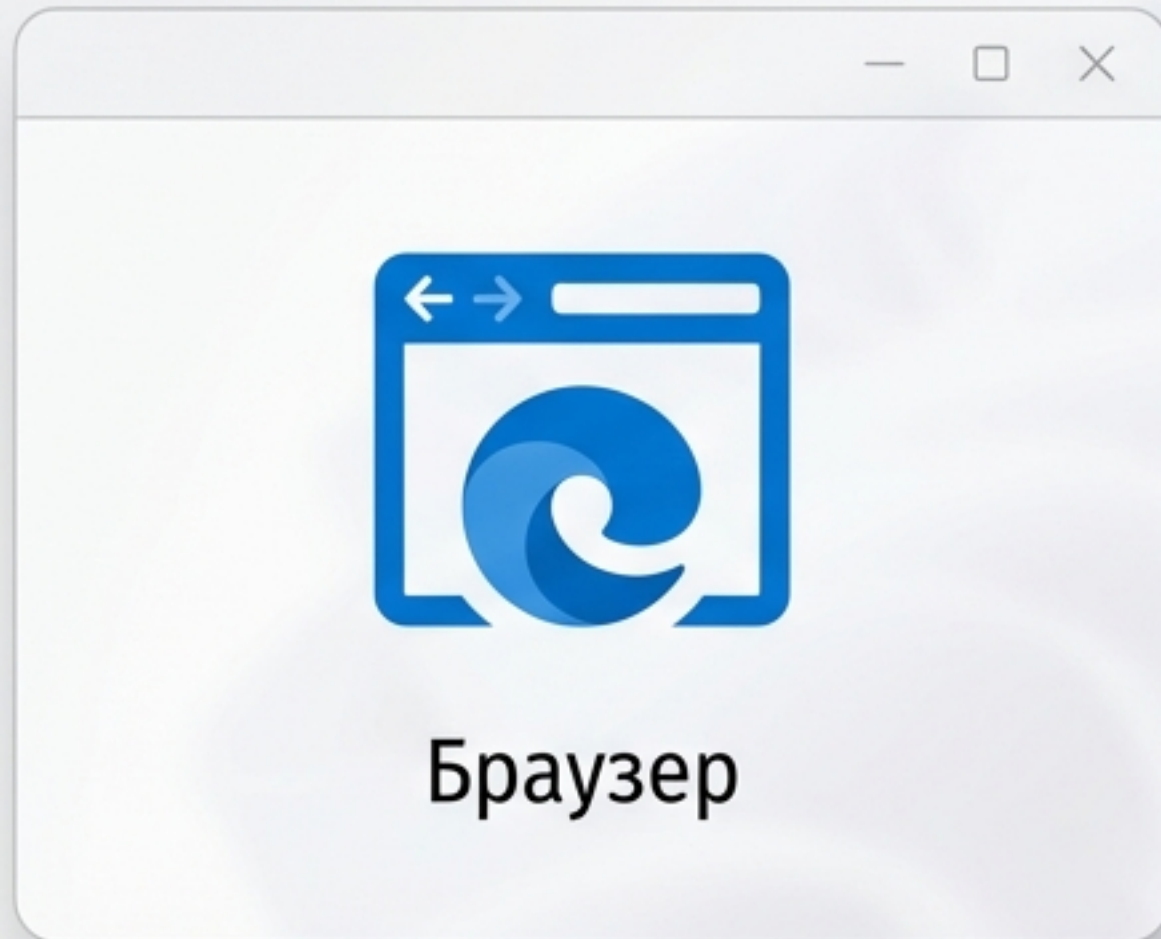
Inter SemiBold

JS: Кто управляет кодом?

Погружение в среду выполнения.

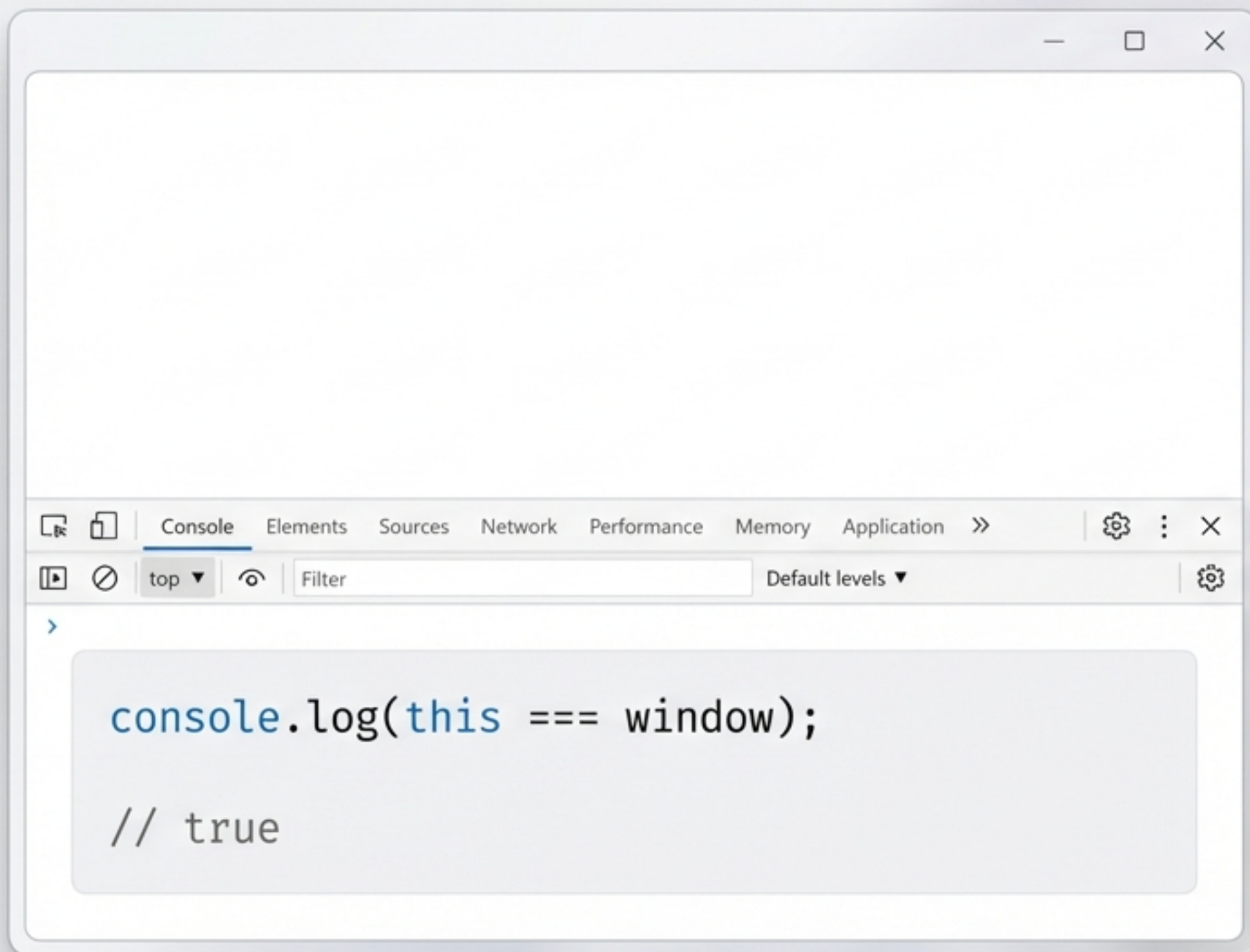
5%

Где исполняется JavaScript?



Один язык, два совершенно разных мира. Правила игры и доступные возможности кардинально меняются в зависимости от среды выполнения.

Мир №1: Браузер и всемогущий `window`.



В браузере глобальный объект `window` — это всё. Он — ваш портал к DOM, Web Platform API (Cookies, Timers, Fetch) и управлению всем, что видит пользователь.

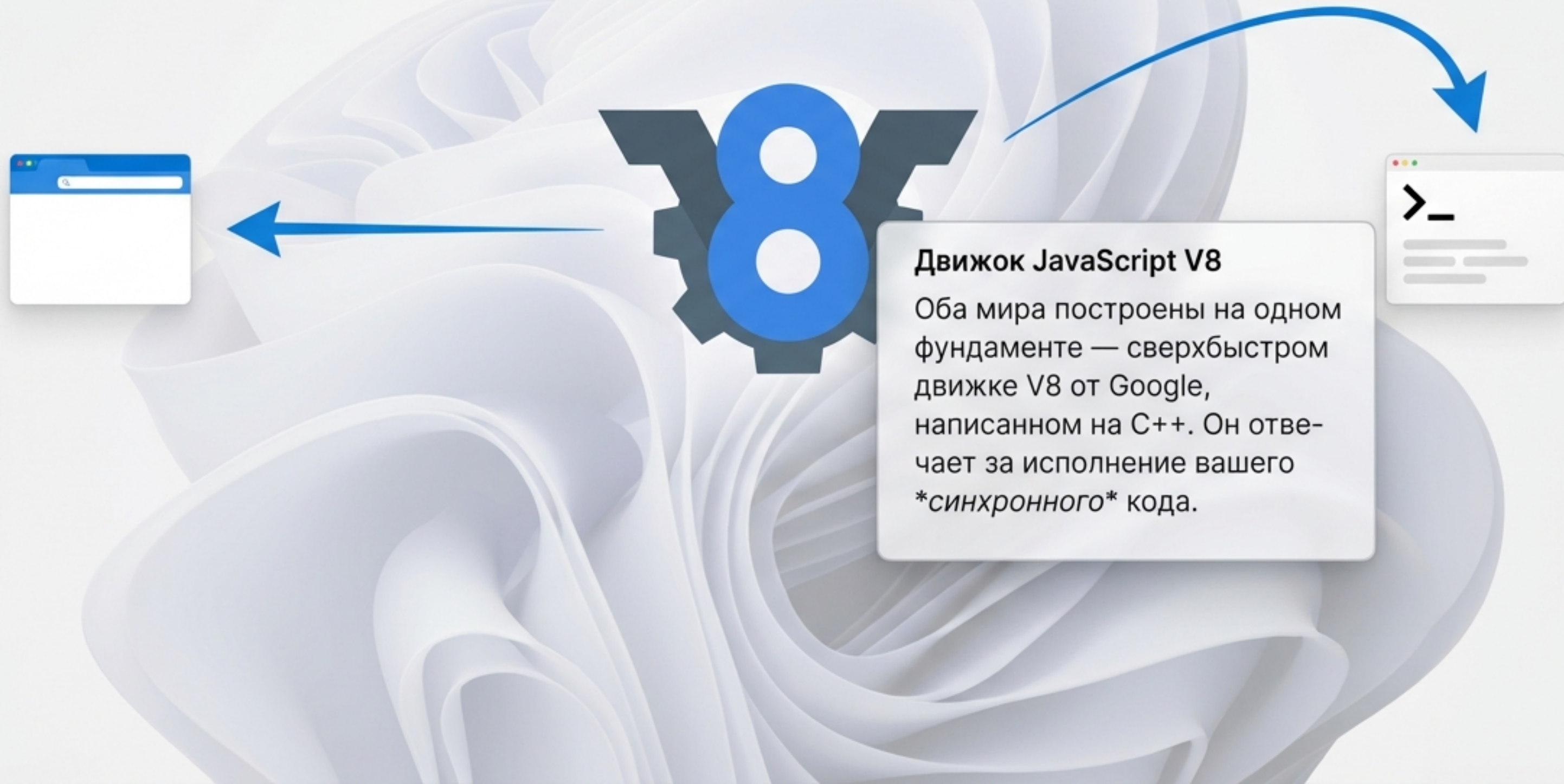
Мир №2: Node.js и системный `global`.

```
// в глобальной области видимости скрипта
console.log(global === this);

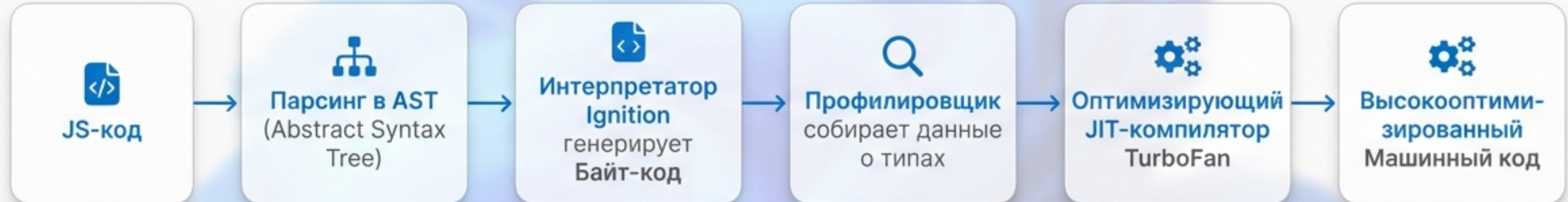
// true
```

В Node.js нет `window` и DOM. Здесь правит объект `global`, предоставляя прямой доступ к ресурсам сервера: файлов: (`fs`), сети (`http`), операционной системе (`os`) и процессам (`process`).

Что у них общего?



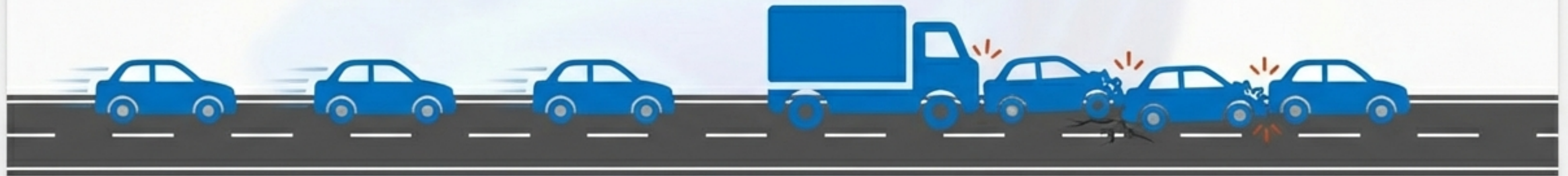
Как V8 превращает код в действие.



V8 не просто интерпретирует, а компилирует код "на лету" (Just-In-Time). Интерпретатор Ignition обеспечивает быстрый запуск, а компилятор TurboFan создает высокопроизводительный машинный код для "горячих" участков, которые выполняются часто.

Проблема: один поток на всех.

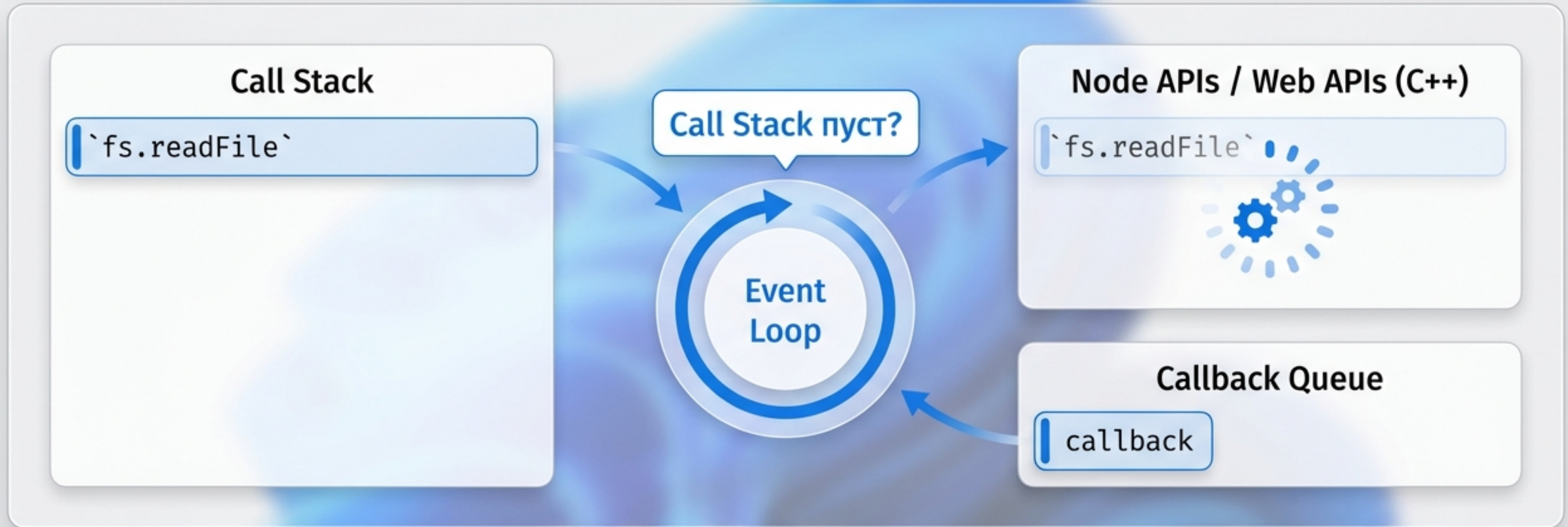
I/O операция: чтение файла



⚠️ Блокировка потока

Движок V8 выполняет код в одном потоке (имеет один Call Stack).
Если задача долгая (сетевой запрос, чтение файла), всё приложение замрет.
Как этого избежать?

Решение: Цикл событий (Event Loop).



Цикл событий — это механизм, который позволяет выполнять неблокирующие операции. Он делегирует долгие задачи среде (API браузера или библиотеке libuv в Node.js) и исполняет их колбэки, когда приходит результат.

Не все задачи равны.



Очередь Микрозадач (Microtask Queue)

Примеры

`Promise.then/catch/finally`` `process.nextTick``,
`queueMicrotask``

Правило

Выполняются **ВСЕ** задачи в очереди после текущей операции, до полного опустошения очереди, прежде чем Event Loop продолжит работу.

Очередь Макрозадач (Macrotask Queue)

Примеры

`setTimeout`, `setInterval`, I/O, рендеринг UI

Правило

Выполняется **ОДНА** задача за одну итерацию (тик) цикла событий.



Порядок выполнения в действии.

```
1 console.log('1. Старт');
2
3 setTimeout(() => {
4   console.log('5. setTimeout (макрозадача)');
5 }, 0);
6
7 Promise.resolve().then(() => {
8   console.log('3. Promise (микрозадача)');
9 });
10
11 console.log('2. Финиш');
```

```
> 1. Старт
> 2. Финиш
// Event Loop: завершена текущая макрозадача
(скрипт), проверяем микрозадачи...
> 3. Promise (микрозадача)
// Event Loop: очередь микрозадач пуста,
начинаем новый тик, проверяем макрозадачи...
> 5. setTimeout (макрозадача)
```

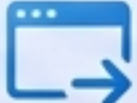

Высший приоритет: `process.nextTick()`.



`process.nextTick()` технически не является частью Event Loop. Его очередь (`nextTickQueue`) обрабатывается немедленно после завершения текущей операции, до очереди микрозадач с промисами.

Это самый быстрый способ выполнить асинхронный код в Node.js, позволяя "вклиниться" перед продолжением цикла событий.

Разные цели — разные циклы событий.

Характеристика	Цикл событий в Браузере 	Цикл событий в Node.js (libuv) 
Основная цель	Плавность интерфейса (UI, 60 FPS) и отзывчивость на действия пользователя.	Максимальная пропускная способность I/O операций на сервере.
Реализация	Интегрирован с движком рендеринга браузера (Blink, Gecko).	Построен на C-библиотеке libuv, оптимизированной для системных вызовов.
Ключевые фазы	Обработка задач, рендеринг, обработка событий.	6 фаз: Timers, I/O callbacks, Poll (опрос I/O), Check (setImmediate).

Инструменты среды: что доступно разработчику?

Категория	Браузер	Node.js
 Манипуляция UI	<code>document</code> , <code>window</code> (DOM API)	Отсутствует
 Доступ к файлам	<code>File API</code> (ограниченный, с разрешения пользователя)	Модуль <code>fs</code> (полный доступ к файловой системе)
 Работа с сетью	<code>fetch</code> , <code>XHR</code> (ограничено политикой Same-origin/CORS)	Модули <code>http</code> , <code>net</code> (создание серверов, без ограничений)
 Системная информация	<code>navigator</code> (ограниченная информация о браузере)	Модули <code>os</code> , <code>process</code> (полная информация о системе и процессе)

Так кто же управляет кодом?

🌐 **Среда выполнения (Браузер / Node.js)**

Исполняет синхронный код

⚙️ **Движок V8**

Call Stack

Дирижирует асинхронным кодом

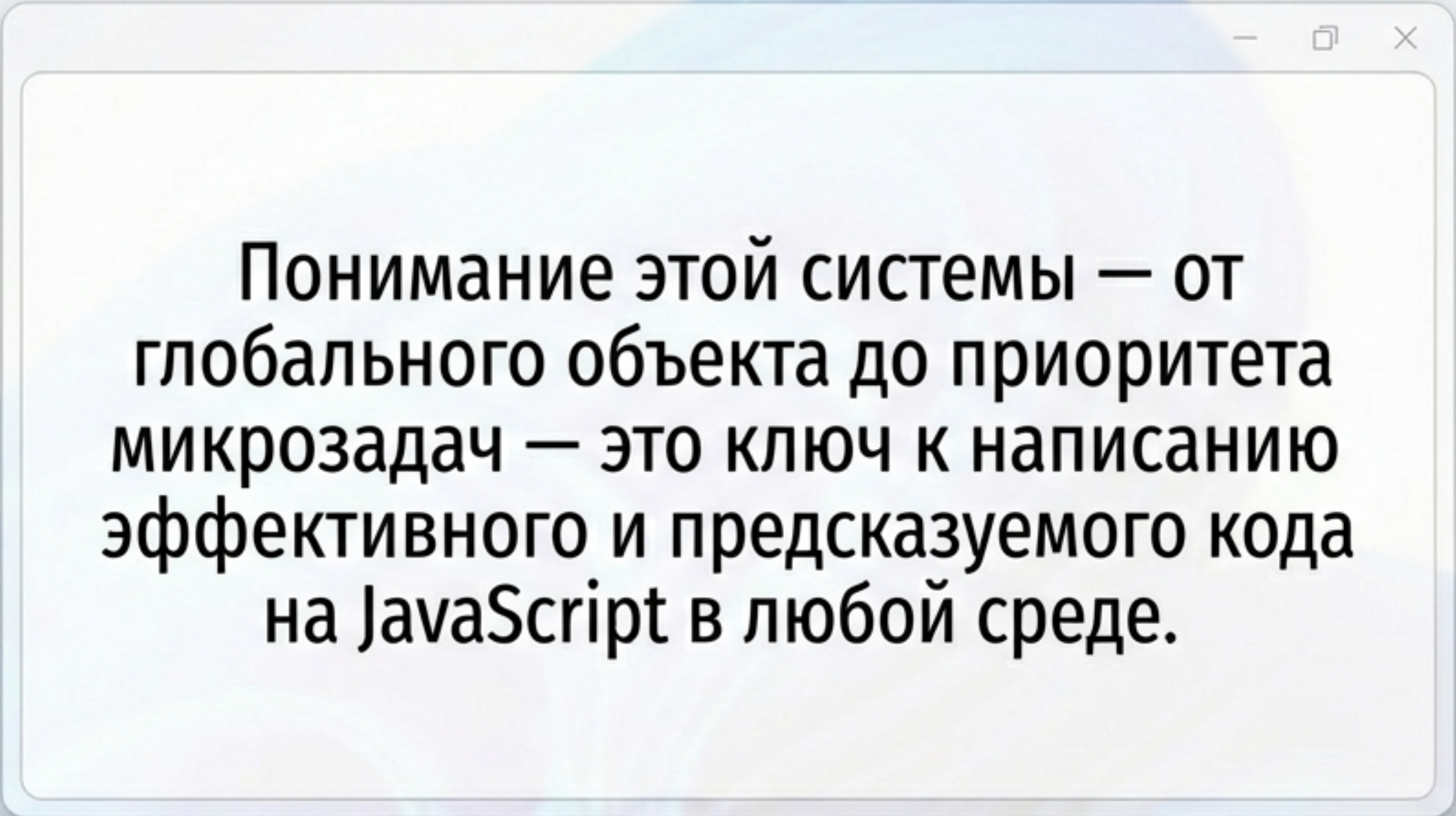
🔄 **Цикл Событий (Event Loop)**

nextTick Queue

Microtask Queue (Promises)

Macrotask Queue

Управляет не кто-то один, а целая система со строгими правилами приоритетов. **Среда** предоставляет возможности, **движок V8** исполняет синхронные инструкции, а **Цикл событий** дирижирует асинхронными операциями, отдавая абсолютный приоритет микрозадам.



Понимание этой системы — от глобального объекта до приоритета микрозадач — это ключ к написанию эффективного и предсказуемого кода на JavaScript в любой среде.

Код управляется правилами. Теперь вы их знаете.